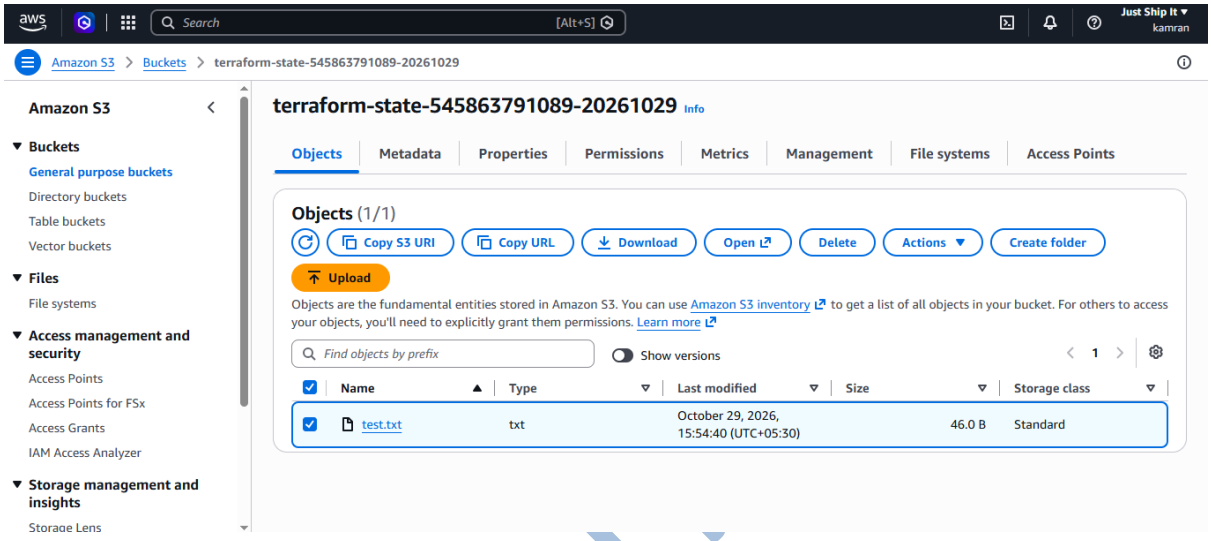
TERRAFORM REMOTE STATE AND STATE LOCKING

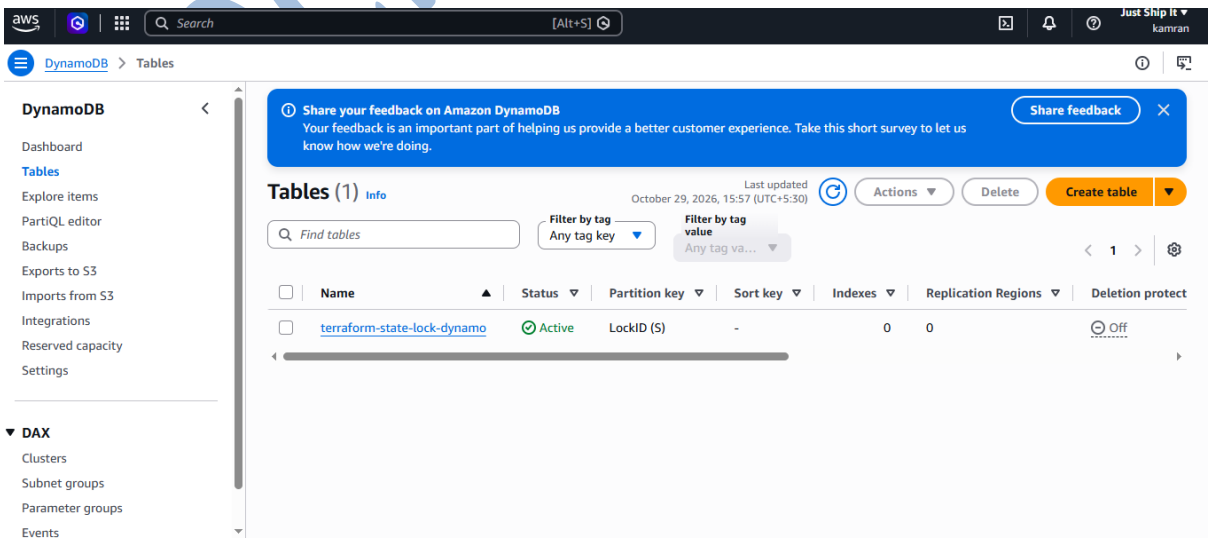
. S3 Remote Backend

- Created an Amazon S3 bucket using Terraform to store the Terraform state file remotely.
- Enabled bucket versioning to maintain previous versions of the state file.
- Enabled server-side encryption and blocked public access.



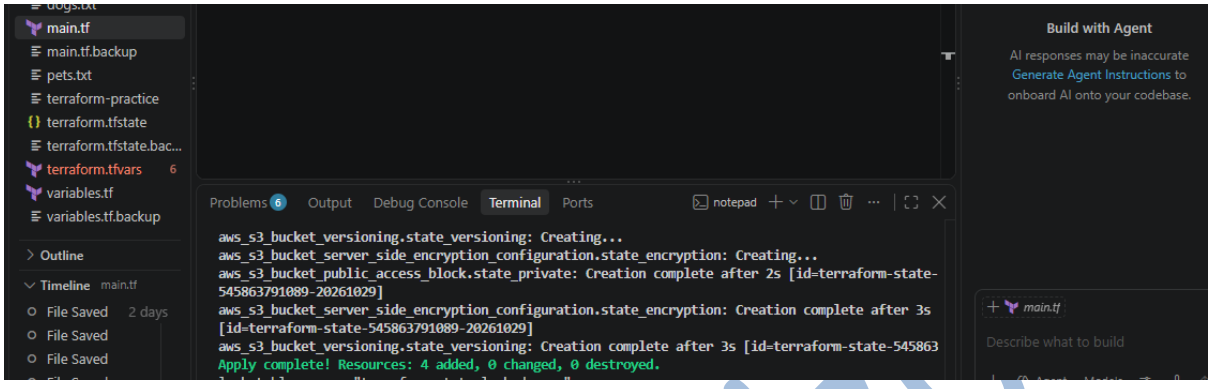
DynamoDB State Locking

- Created a DynamoDB table named `terraform-state-lock-dynamo`.
- Configured `LockID` as the string partition key.
- Prepared the table for Terraform state locking.



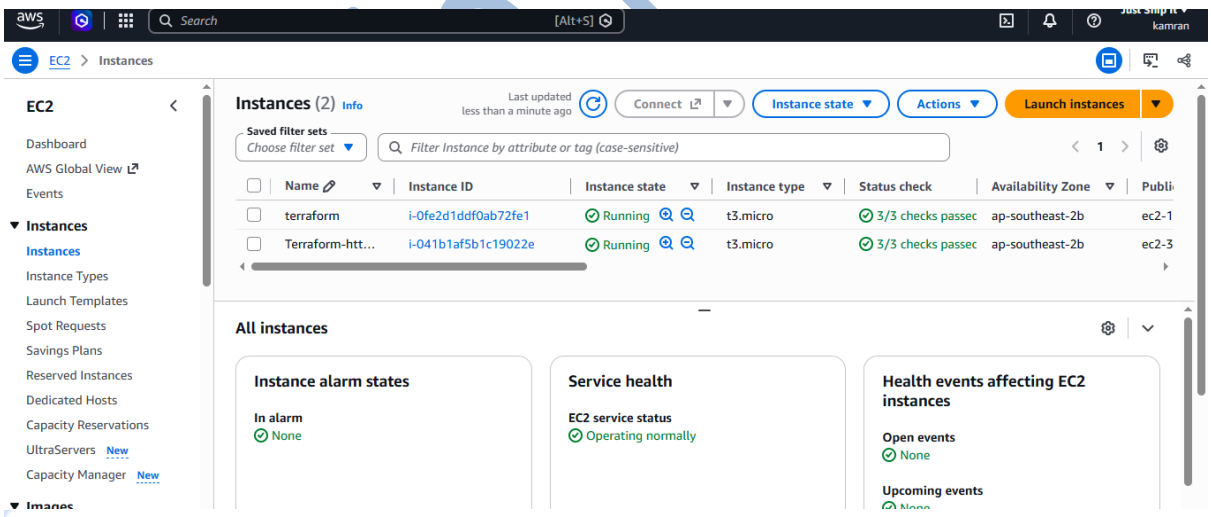
Terraform Provisioning

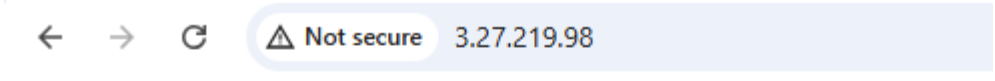
- Initialized and validated the Terraform configuration.
- Reviewed the execution plan before applying changes.
- Resolved IAM permission issues and the globally unique S3 bucket-name conflict.



Terraform Configuration

- Configured the AWS provider in the `ap-southeast-2` region.
- Used an AWS AMI data source to retrieve a suitable Amazon Linux image.
- Created a security group allowing inbound HTTP traffic on port `80`.
- Provisioned an EC2 instance using Terraform.
- Used `user_data` to install and start the Apache HTTP server (`httpd`).
- Configured Terraform outputs to display the EC2 public IP and website URL.

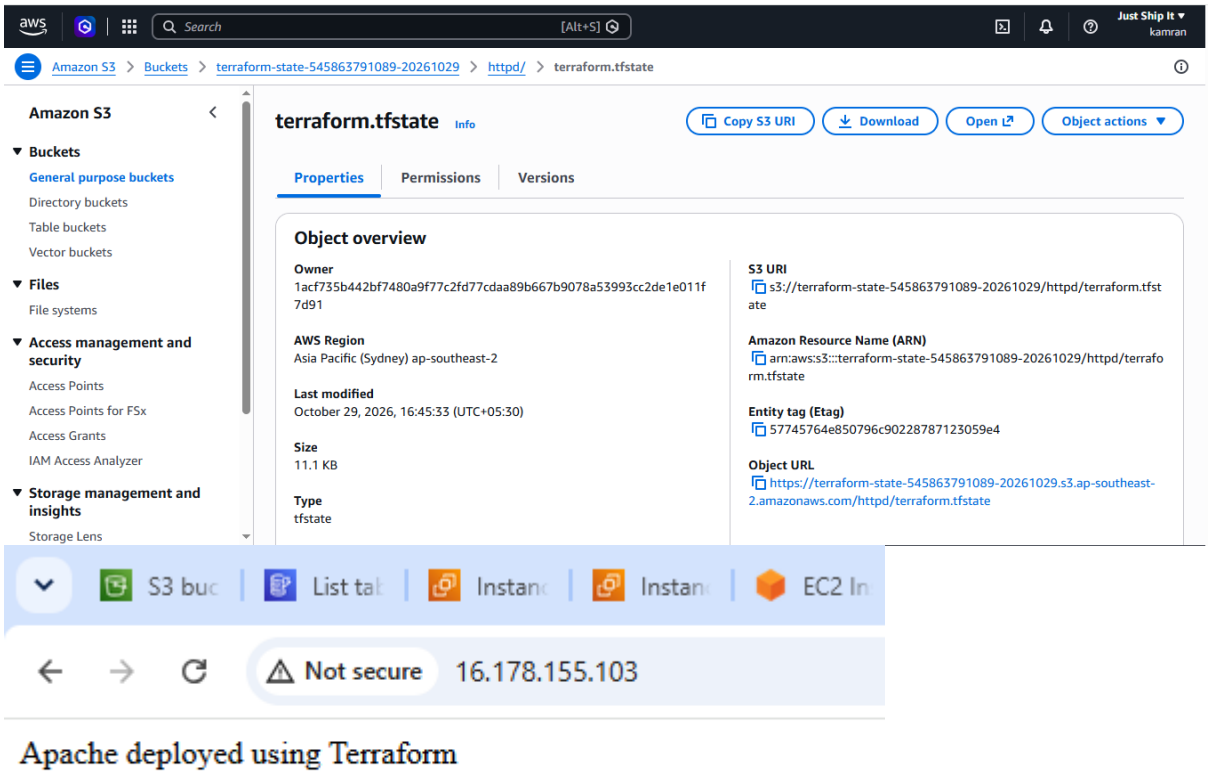






Set up S3 as backend for task 3

- Created an Amazon S3 bucket to store Terraform state files remotely.
- Configured the S3 backend in the Terraform configuration.
- Specified the AWS region as `ap-southeast-2`.
- Configured the state file key as `httpd/terraform.tfstate`.
- Enabled encryption for the remote state file.
- Migrated the existing Terraform state to the S3 backend.
- Configured DynamoDB for state locking as required by the assignment.
- Verified the remote state file in the S3 bucket.



DynamoDB Locking in Terraform

- Created an Amazon DynamoDB table named `terraform-state-lock-dynamo`.
- Configured `LockID` as the partition key with the `String` data type.
- Configured the DynamoDB table in the Terraform S3 backend.
- Enabled state locking to help prevent concurrent Terraform operations from modifying the state simultaneously.
- Verified that Terraform successfully acquired and released the state lock during execution.
- Used the S3 backend to store Terraform state remotely and DynamoDB for state locking.

```
      }

    ~ root_block_device (known after apply)
    - root_block_device {
        - delete_on_termination = true -> null
        - device_name           = "/dev/xvda" -> null
        - encrypted             = false -> null
        - iops                  = 3000 -> null
        - tags                  = {} -> null
        - tags_all              = {} -> null
        - throughput            = 125 -> null
        - volume_id             = "vol-0e8682ba5b6794ed1" -> null
        - volume_size           = 8 -> null
        - volume_type           = "gp3" -> null
          # (1 unchanged attribute hidden)
      }

    ~ secondary_network_interface (known after apply)
  }

Plan: 1 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  ~ instance_public_ip = "16.178.155.103" -> (known after apply)
  ~ website_url        = "http://16.178.155.103" -> (known after apply)
```

